PENETRATION TESTING & ASSESSMENT REPORT

Apache Tomcat Web Application Assessment

Enterprise Target	Omniseck Cyber
Platform	TryHackMe
Student / Analyst	Sirapu Durga
TryHackMe Profile	tryhackme.com/p/sirapudurgareddy
Assessment Date	5 October 2026

Report scope: This report documents a controlled, educational TryHackMe lab assessment. The findings and actions described should be interpreted only in the context of the authorized lab target.

Prepared for educational and portfolio documentation.

Executive Summary

This document serves as the technical security assessment report for the Apache Tomcat infrastructure under **Omniseck Cyber** in a TryHackMe lab. The described scope includes web surface discovery, review of authentication and authorization weaknesses in the Apache Tomcat Manager interface, authorized testing of application deployment controls, and verification of access obtained during the lab. The original notes describe use of a Web Application Archive (**.war**) payload and a listener to test whether server-side code execution was possible.

> **Important validation note:** The supplied draft contains multiple target IP addresses (10.81.184.81, 10.81.178.98, and 10.81.123.189). Confirm which address belongs to the target and which belongs to the testing machine before submitting this report. The evidence sections were left as placeholders because no screenshots were included with the supplied text.

Assessment Overview

Area	Summary
Target service	Apache Tomcat web interface, noted on TCP port 8080
Primary issue described	Weak/default credentials for the Tomcat Manager interface
Potential impact	Unauthorized application deployment and possible server-side code execution
Evidence status	Screenshots to be inserted by the analyst

10-Step Technical Execution & Walkthrough

Step 1: Target Room & Environment Initialization

The assessment commenced on the TryHackMe platform against the target instance provisioned for the lab. The analyst checked that the lab machine was active and that network connectivity was available.

Technical detail: The supplied notes list target IP addresses `10.81.184.81` and `10.81.178.98`. Confirm the correct target address before final submission.

Evidence: Insert screenshot showing the TryHackMe room and active target details.

Step 2: Web Surface Discovery & HTTP Enumeration

Initial service enumeration identified an HTTP web interface associated with Apache Tomcat on port 8080.

Technical detail: The URL `http://10.81.178.98:8080` reportedly displayed the Apache Tomcat/8.5.5 landing page.

Evidence: Insert screenshot of the Tomcat landing page and the visible address bar.

Step 3: Administrative Endpoint Probing & IAAA Vulnerability Identification

An unauthenticated request to the restricted `/manager/html` endpoint reportedly returned HTTP 401 Unauthorized, indicating that authentication was required.

Technical detail: The response reportedly referenced the `tomcat-users.xml` configuration file and roles such as `manager-gui`.

Evidence: Insert screenshot of the authentication challenge or 401 response. Redact any unrelated secrets.

Step 4: Authentication Attempt Using Default Credentials

The analyst tested the lab's configured credentials against the HTTP Basic Authentication prompt for the Tomcat Manager endpoint.

Technical detail: The supplied draft records the credential pair `tomcat : s3cret`. Include this only if it matches the actual lab evidence.

Evidence: Insert screenshot of the authentication prompt or successful lab result without exposing credentials unnecessarily.

Step 5: Authenticated Access to Tomcat Manager Dashboard

The supplied notes state that authentication succeeded and the Tomcat Manager dashboard became accessible.

Technical detail: Access to the manager interface may permit viewing deployed applications and managing deployment actions, depending on the assigned role.

Evidence: Insert screenshot of the Manager dashboard with the browser URL visible.

Step 6: Payload Preparation

The original notes describe preparing a Web Application Archive (.war) for an authorized lab validation of server-side execution. This report records the test objective without reproducing payload-generation commands.

Technical detail: Record the payload filename, test timestamp, and any relevant hash in the final report if these were captured during the lab.

Evidence: Insert a screenshot showing the prepared artifact and its filename, with sensitive details redacted.

Step 7: Listener Configuration

The notes describe configuring a Metasploit handler on the testing machine to observe whether the lab application could initiate a callback.

Technical detail: The supplied notes list `10.81.123.189` as the callback address. Verify that this is the testing machine's lab VPN address and record the configured listening port.

Evidence: Insert screenshot of the handler configuration, redacting unrelated session details.

Step 8: Handler Readiness Verification

The analyst checked that the framework initialized and that the handler was ready before testing the application.

Technical detail: The supplied draft records framework version `v6.4.55-dev`. Confirm the version from the actual terminal output.

Evidence: Insert screenshot showing the framework version and handler readiness.

Step 9: Application Deployment & Trigger Validation

The supplied notes describe uploading the test archive through Tomcat Manager and requesting the application context to validate whether deployment and execution were possible.

Technical detail: The recorded application URL is `http://10.81.178.98:8080/shell/`. Confirm the address and outcome against the screenshots.

Evidence: Insert screenshot of the deployed application entry and the request result.

Step 10: Shell Callback Verification

The original notes state that requesting the deployed application was followed by an interactive shell session. This outcome should be reported only if a matching session or terminal screenshot confirms it.

Technical detail: The draft also references `http://10.81.178.98:8080/shell/shell.war`. A request to a WAR file alone does not prove code execution; corroborate the claim with handler/session evidence.

Evidence: Insert screenshot of the confirmed session and a minimal verification command, avoiding disclosure of secrets.

Findings & Risk

Finding 1 — Weak or Default Tomcat Manager Credentials
If the recorded login is confirmed by evidence, credentials that are predictable or unchanged from defaults may allow an unauthorized user to access administrative deployment features. **Potential severity:** High, depending on exposure, privileges, and network reachability.

Finding 2 — Application Deployment Could Enable Server-Side Code Execution
If the lab evidence confirms that a deployed application produced a callback, the deployment interface and service permissions created a path from manager access to code execution. The exact impact depends on the Tomcat service account's privileges and the environment's controls.

> **Evidence limitation:** These findings are based on the supplied narrative, not independently reviewed screenshots or logs. Treat them as provisional until the evidence is attached and the IP addresses and outcomes are verified.

Defensive Remediation & Hardening Recommendations

1. Eliminate default credentials. Replace default or weak entries in `conf/tomcat-users.xml` with unique, strong credentials and rotate any credentials exposed during testing.

2. Restrict administrative access. Limit `/manager/html` and `/host-manager/html` to approved management hosts or a trusted administration network. Use network controls and Tomcat access restrictions.

3. Disable unnecessary manager interfaces. Remove or disable the Tomcat Manager application when remote deployment is not required in production.

4. Enforce least privilege. Run Tomcat under a dedicated unprivileged operating-system account with narrowly scoped filesystem and service permissions.

5. Patch and monitor. Upgrade unsupported or vulnerable Tomcat versions, review access and deployment logs, alert on unexpected application deployments, and investigate unusual outbound connections.

6. Validate and document. Retest after remediation and retain dated screenshots, relevant logs, exact target details, and a clear separation between target and testing-machine addresses.

Conclusion

The described lab exercise illustrates how weak administrative credentials and an exposed application deployment interface can increase the risk of server compromise. The most important corrective actions are to remove default credentials, restrict management access, disable unnecessary deployment functionality, run services with least privilege, and verify remediation with recorded evidence.

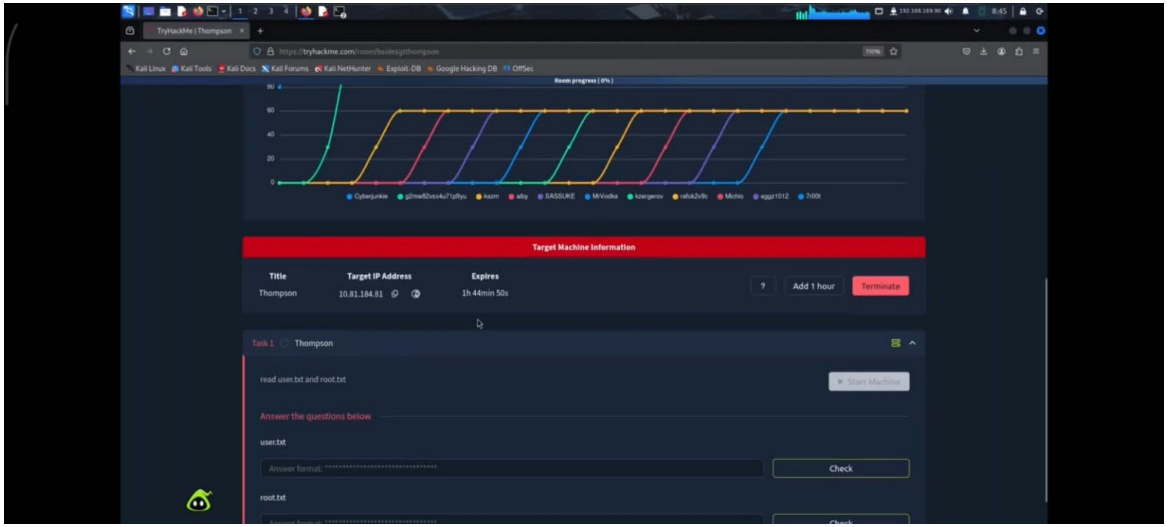
Appendix: Evidence Checklist

- ■ TryHackMe room and active target screenshot
- ■ Tomcat landing page on port 8080
- ■ Unauthenticated Manager access response

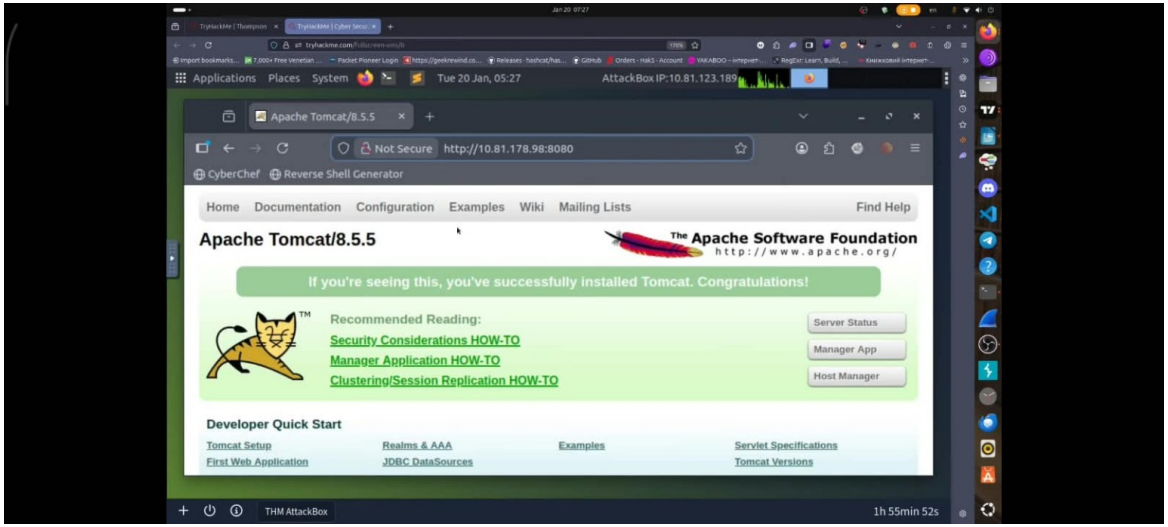
- Manager authentication result and dashboard
- Prepared test artifact details
- Handler readiness and callback/session confirmation
- Deployment listing and application request result
- IP address confirmation: target vs. testing machine

Appendix A — Screenshot Evidence

Evidence 1 — TryHackMe Thompson room and active target details

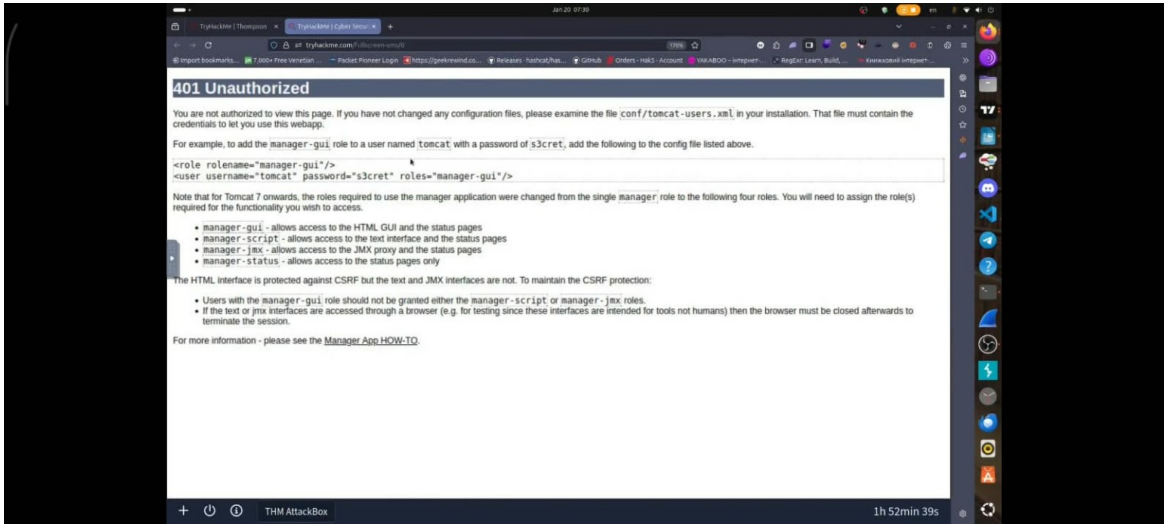


Evidence 2 — Apache Tomcat 8.5.5 landing page on port 8080

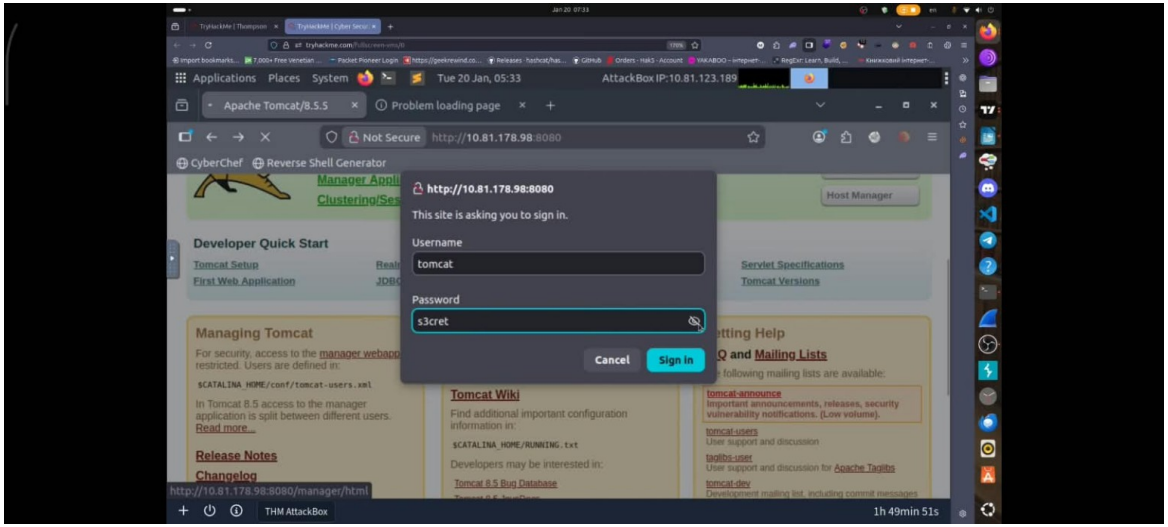


Appendix A — Screenshot Evidence

Evidence 3 — Tomcat Manager endpoint returns HTTP 401 Unauthorized

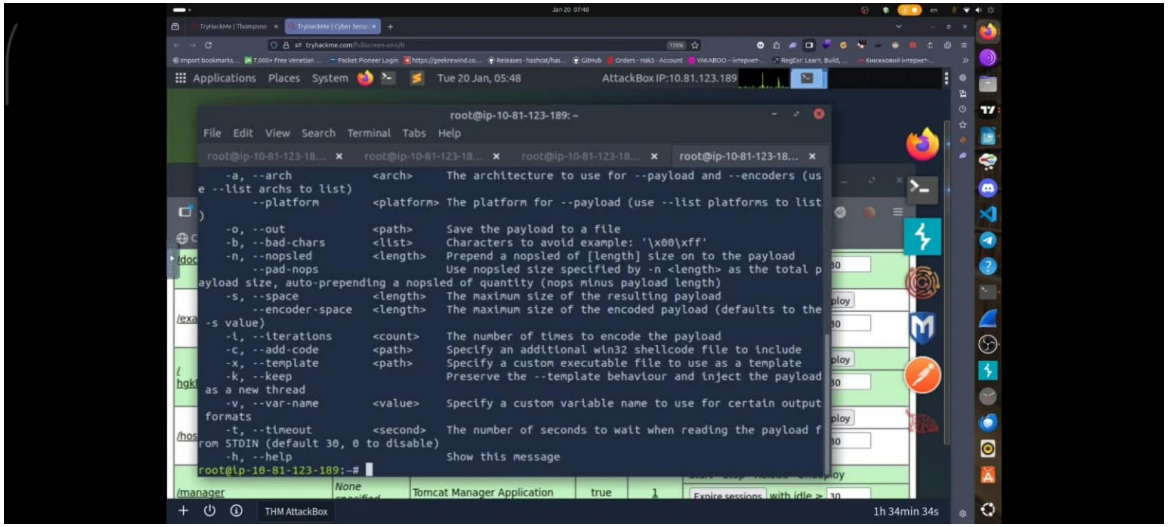


Evidence 4 — HTTP Basic Authentication prompt shown for the Manager interface

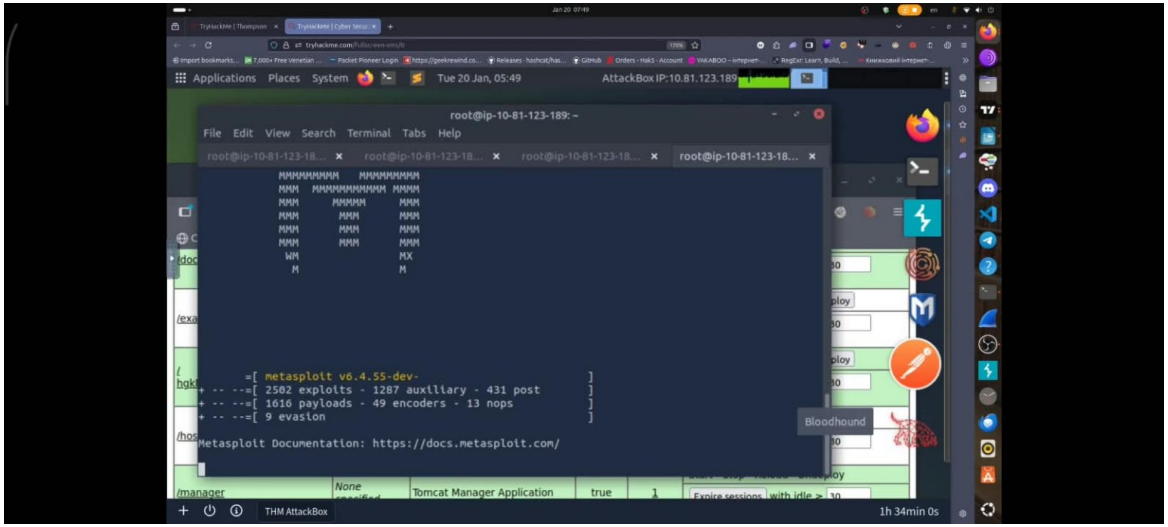


Appendix A — Screenshot Evidence

Evidence 5 — Metasploit command-line help / payload preparation context

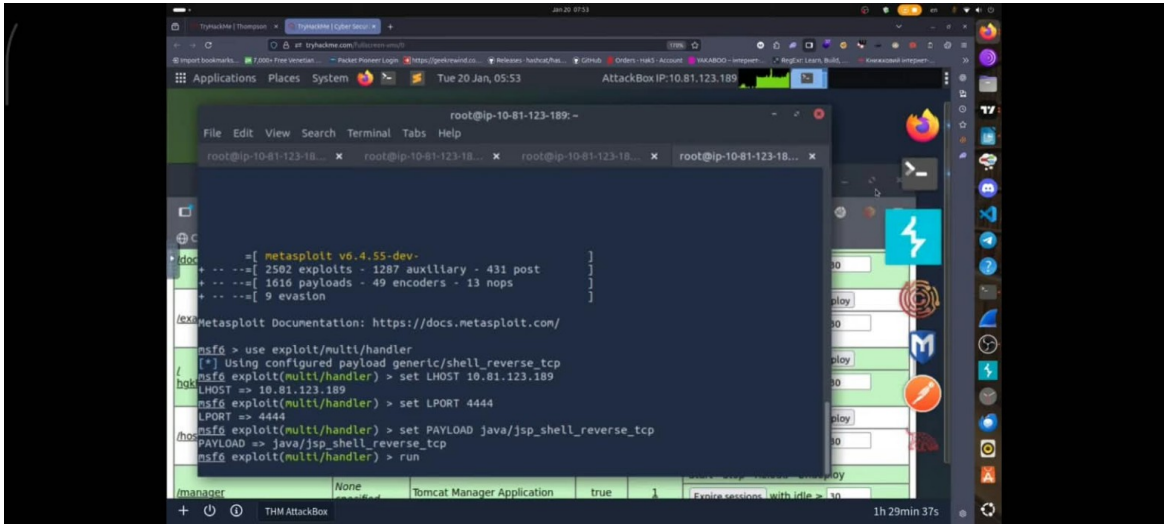


Evidence 6 — Metasploit framework startup

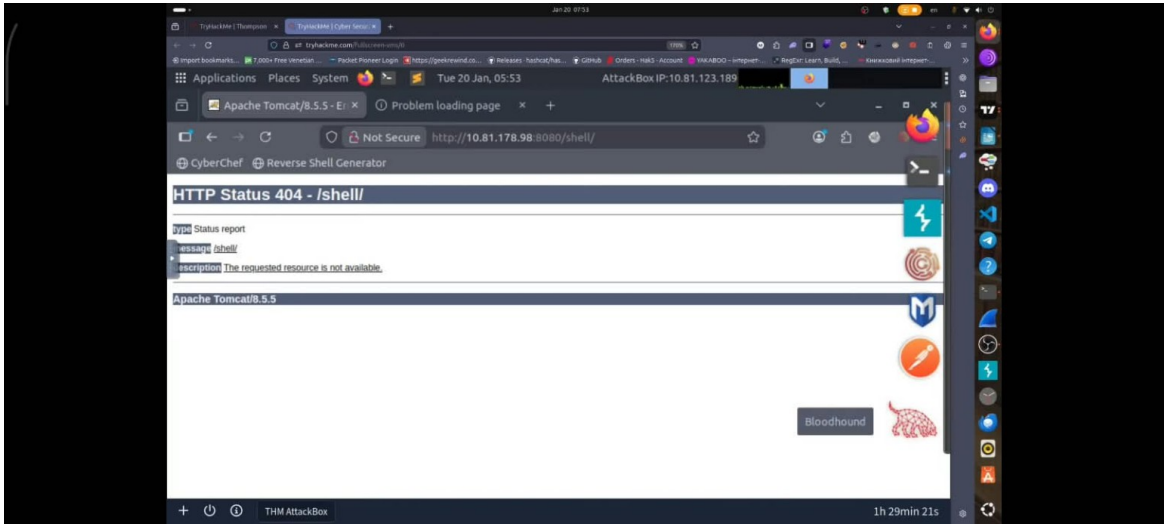


Appendix A — Screenshot Evidence

Evidence 7 — Reverse TCP handler configuration shown in the lab



Evidence 8 — Request to /shell/ returns HTTP 404; deployment is not confirmed



Appendix A — Screenshot Evidence

Evidence 9 — curl request confirms HTTP 404 for /shell/

